Techniques for improving reliability of DNN algorithms and accelerators

Sparsh Mittal

Acronyms

- TE =timing error
- BER = bit error rate
- ECC = error-correcting code
- TPU = (Google's) tensor processing unit
- SECDED = single error correction, double error detection
- SDC = silent data corruption
- DUE = detected unrecoverable error
- TMR = triple modular redundancy
- FP = floating point, FxP = fixed point

Reliability of DRAM in a DNN accelerator

Observations on weights and activations

- **Weights:** 99.5% of weights of GoogleNet, AlexNet and ResNet50 are in the range [2^{-13}, 2^{-2}] =>positions {30,29,28} of FP32 are {0,1,1}

2^k for $k \in [11, -15]$ and 31st-23rd bits of corresponding FP32 representation

2^k	31st-23rd bits	2^k	31st-23rd bits	2^k	31st-23rd bits	2^k	31st-23rd bits	2^k	31st-23rd bits
2^{11}	0 10001010	2^5	0 10000100	2^{-1}	0 01111110	2^{-7}	0 01111000	2^{-13}	0 01110010
2^{10}	0 10001001	2^4	0 10000011	2^{-2}	0 01111101	2^{-8}	0 01110111	2^{-14}	0 01110001
2^9	0 10001000	2^3	0 10000010	2^{-3}	0 01111100	2^{-9}	0 01110110	2^{-15}	0 01110000
2^8	0 10000111	2^2	0 10000001	2^{-4}	0 01111011	2^{-10}	0 01110101		
2^7	0 10000110	2^1	0 10000000	2^{-5}	0 01111010	2^{-11}	0 01110100		
2^6	0 10000101	2^0	0 01111111	2^{-6}	0 01111001	2^{-12}	0 01110011		

- **Activations** of GoogleNet and AlexNet are in range [2^1,2^{11}] => bits {30,29,28} of FP32 are {1,0,0}.
- 99.9% activations of ResNet50 fall in range [2^{-12},2^{11}].
-

Nguyen et al. DAC 2019

Selective ECC for protecting DRAM

- Reason behind above observations: In training, regularization is used for limiting the range of weights to [-1,1]. By comparison, activations are obtained from multiplication => they are generally larger than 1.
- Based on these observations, they propose two schemes

	Scheme-1	Scheme-2
Correction data stored in bits	{30 29 28}	{3 2 1 0}
Bits which receive protection	{31 27 26 25}	{31 30 29 28 27 26 25 24 23}

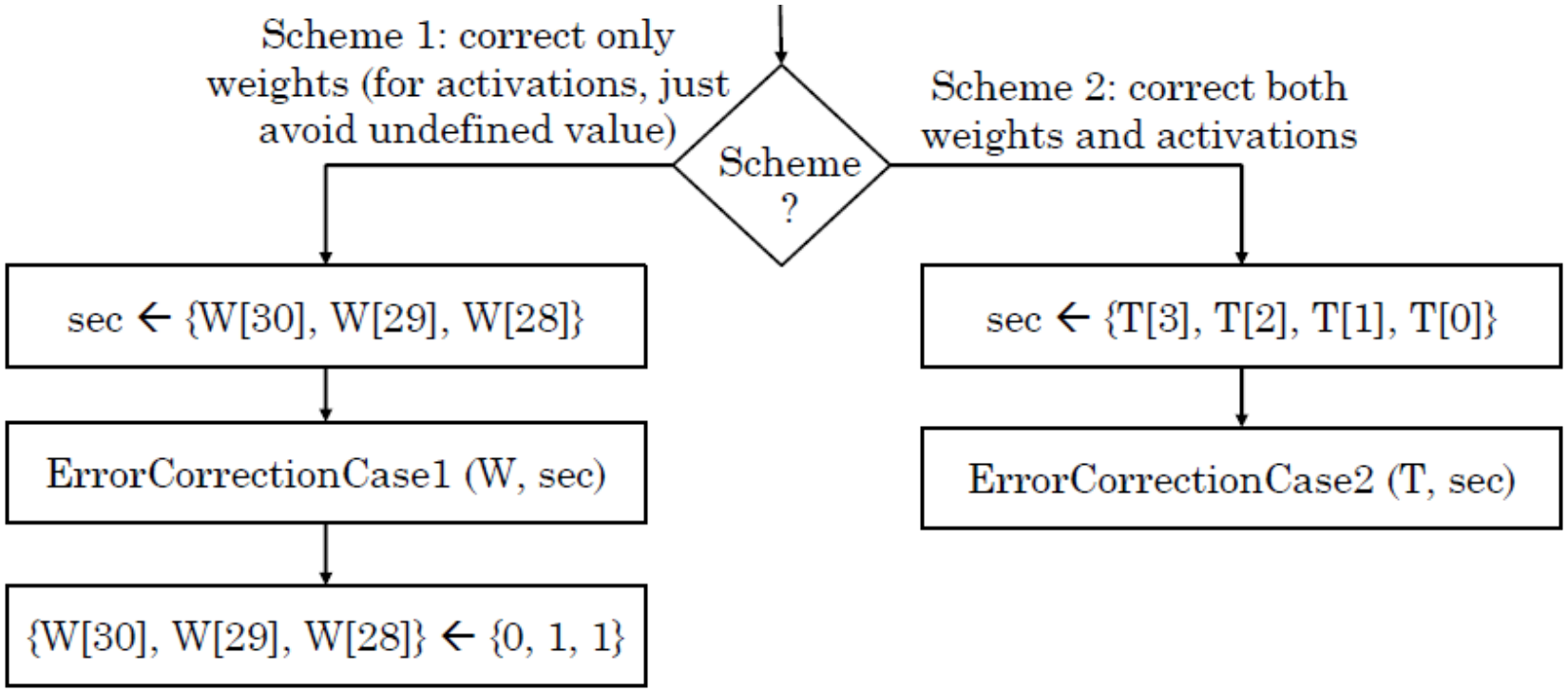
Nguyen et al. DAC 2019

Scheme 1

- Hamming(7,4) code bits are stored in bits {30,29,28} and they are used for correcting {31, 27, 26, 25} bits of FP32 weight. After error correction, value of {30,29,28} bits are assigned to {0,1,1}
- In scheme-1, errors can happen in only two bits of the exponent (24th and 23rd). This error changes weights by at most 2^2, which can be alleviated by training.
- Scheme-1 does not protect activations since they are much more resilient than the weights and they stay in memory for short period. Only if their value becomes undefined, their 30th bit is changed to 0 to remedy this error.

Scheme 2

- It stores Hamming(13,9) bits in four LSBs of FP32, and they are used to protect bits 31st to 23rd of FP32. In this scheme, both weights and activations are protected

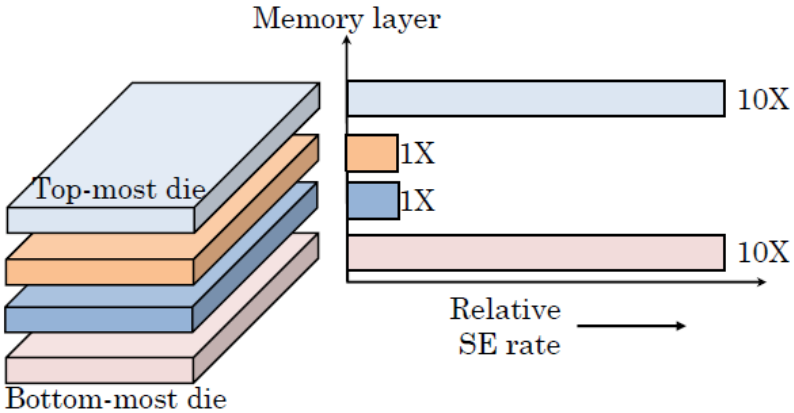


Nguyen et al. DAC 2019

Reliability of stacked memory such as “high-bandwidth memory” (HBM)

Reliability profile of stacked memory

- Top-most die: has high SE rate due to alpha particles arising from upper packaging material
- Bottom-most die: has high SE rate due to the heat released from the interposer and logic die



Final SE rate profile

Kim et al. DAC 2019

Observations

- All weights of AlexNet are below 1 => (30th,29th)bits =(0,1)
- 0→1 flip has higher impact than 1→0 flip
- Apart from the 30th bit, they find the second 0 bit in the higher bits of exponent, which is termed as ``second-zero location bit'' (SZLB)
- Flipping of SZLB has a high impact on accuracy.
- SZLB of most weights of different DNNs
  - 26th bit for VGG16 and AlexNet,
  - 25th bit for pruned VGG16 and pruned AlexNet.
  - 27th bit for remaining weights

Kim et al. DAC 2019

Protection technique

- 30th bit is called “most unreliable bit" (MUB)
- 27th, 26th and 25th bits: “second-most unreliable bit" (SMUB).

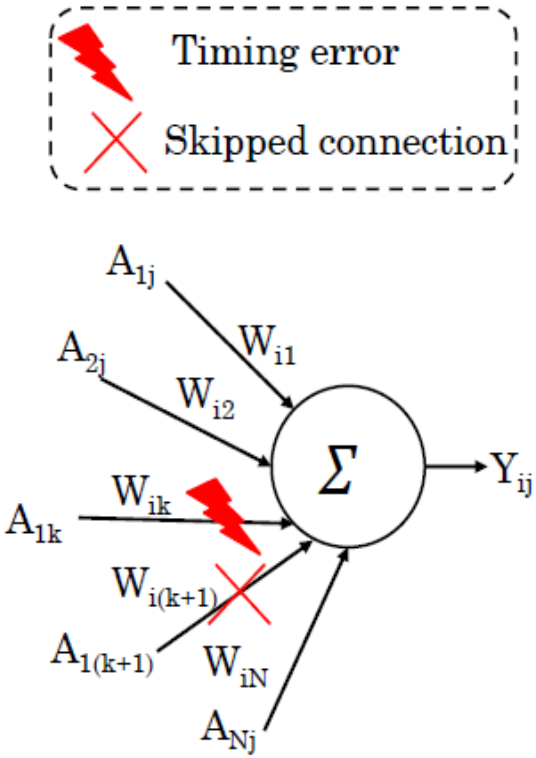
Observations	Terminology	Protection technique
Weights of AlexNet are below 1 $\Longrightarrow$ W[30]=0 and W[29]=1	30th bit is termed MUB	Mask MUB
SZLB is 26th bit for most weights of VGG16 and AlexNet and SZLB is 25th bit for most weights of pruned VGG16 and pruned AlexNet. For the remaining weights, SZLB is 27th bit.	27th, 26th and 25th bits are termed as SMUB	Store SMUB in normal dies
Remaining bits are not critical	Remaining bits are non-SMUB	Store non-SMUB bits in vulnerable dies

Kim et al. DAC 2019

Techniques for Voltage-Scaled Accelerators

TE-drop technique

- Uses Razor flip-flops for detecting TEs
- Weights of a DNN are biased towards small values=> each MAC operation makes an only a small contribution to the output of a neuron.
- On a TE in a MAC, TE-drop technique steals subsequent clock cycle from its downstream MAC for correctly finishing its own update to “partial sum"
- Also skips update of downstream MAC



Zhang et al. DAC 2018

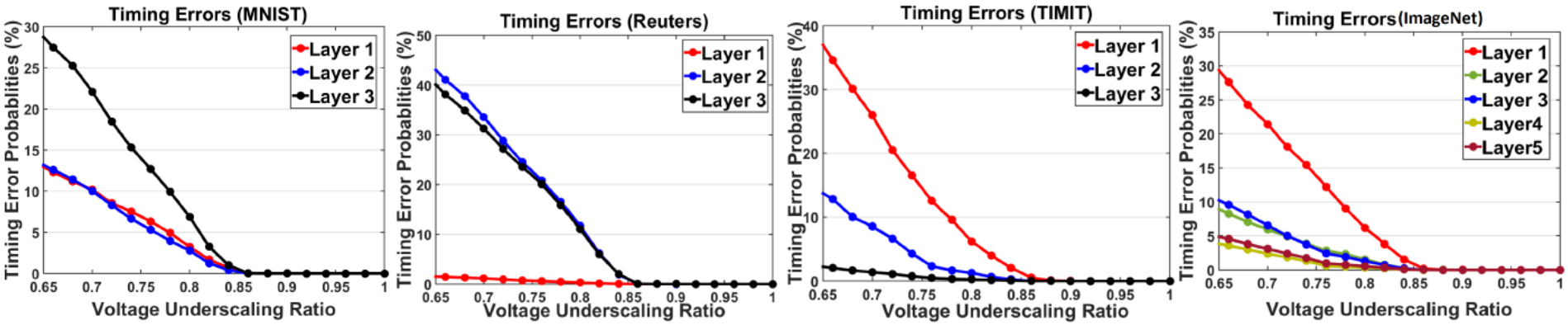
Similarity with dropout and dropconnect

- In effect, TE-drop technique randomly drops some connections between two neurons where the randomness comes due to TEs.
- For avoiding overfitting, techniques such as dropout and dropconnect are anyway used in DNN training which randomly drop neurons and connections.
- TE-drop technique is similar to these techniques and allows operating at more than 10% timing TE rates without loss in accuracy.

Zhang et al. DAC 2018

Layer-specific voltage underscaling

- Different layers show different TE rates on using the same voltage underscaling ratio.



- **Idea:** apply different degree of voltage underscaling to different layers such that the TE rate contribution of all the layers is nearly the same. This technique shows lower TE rate than a simple technique that applies uniform voltage underscaling to all the layers.

Zhang et al. DAC 2018

Limitations of TE-drop technique

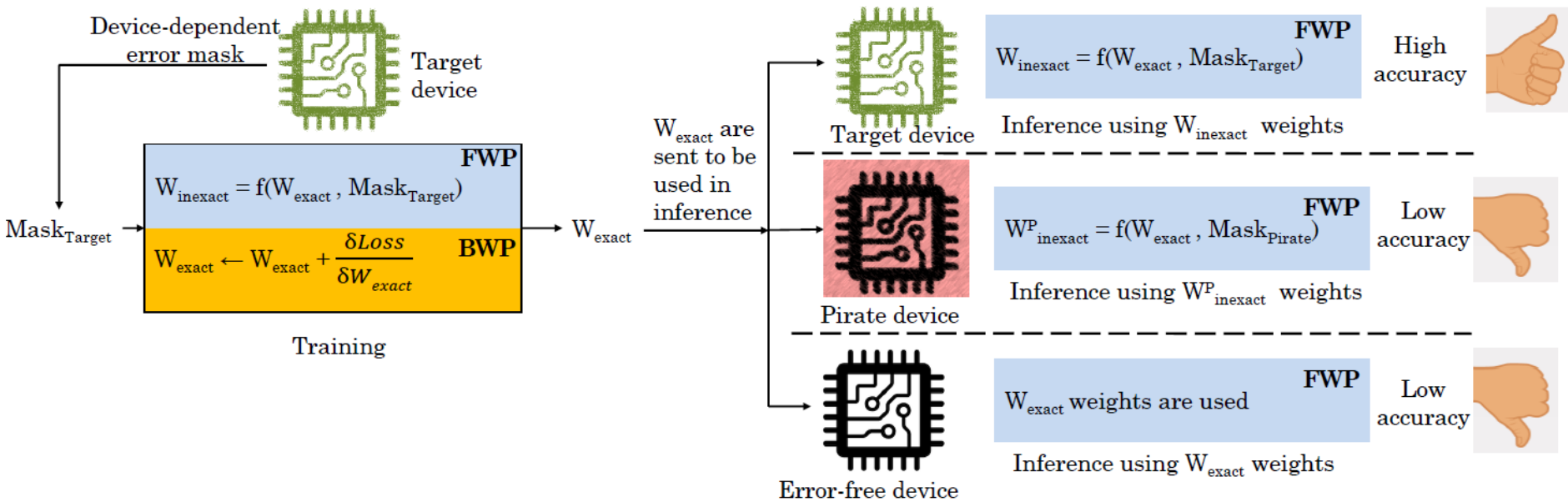
- Using it for mitigating a TE in the last row of MACs leads to large performance loss.
- Values of partial sums are highest at the bottom of systolic array ➔ the impact of TEs is also highest in the last row of MACs.
- Further, with increasing fault rate, skipping update of some MACs reduces the final accuracy significantly.

Security implications of reliability issues

Protecting DNN Model IP

- DNN model is a valuable IP for a company
- Zhao et al. propose a technique for achieving IP protection by leveraging the uniqueness of error-mask of DRAM in a DNN accelerator
- During training, their technique maintains two groups of weights: inexact weights which are used for FWP and exact weights which are for weight update.
- They apply memory error mask at the time of generating the inexact weights and add a batch norm layer to recover accuracy loss.

Zhao et al. ICCAD 2017



Zhao et al. ICCAD 2017

Protecting DNN Model IP

- After completion of training, exact weights are sent to the NN accelerator (“target device”).
- Target device has its error mask, and hence, it can obtain inexact weights from exact weights, which provide high accuracy.
- Assume an attacker using exact weights on any other non-ideal or ideal device. She cannot get high accuracy!

Zhao et al. ICCAD 2017

To generate device-dependent error masks …

- …they reduce tRAS (row active time) parameter, e.g., from 35ns, it is reduced to 11ns.
- This leads to device-dependent restore errors due to process variation.
- Due to restore errors, some cells may map as 0 and others as 1.
- Hence, the error mask has two submasks, which denote the cells stuck at 0 and 1, respectively.

References

- S. Mittal et al., “A Survey of Techniques for Modeling and Improving Reliability of Computing Systems,” IEEE TPDS, 2015.
- S. Mittal et al., “A Survey of Techniques for Improving Error-Resilience of DRAM,” JSA 2018
- S. Mittal, “A Survey Of Architectural Techniques for Near-Threshold Computing,” ACM JETC, 2015.
- S. Mittal, “A Survey of Soft-Error Mitigation Techniques for Non-Volatile Memories,” Computers, 2017